MATERIAL DE APOIO PRÁTICO

Fundamentação prática de Git, GitHub e Bash

Comandos básicos, fluxo de trabalho e conceitos do GitHub para acompanhar a aula.

Bash	Git	GitHub
Navegação e arquivos	Commits, branches e merge	Clone, push, pull e colaboração

Sumário

1. Comandos básicos do Bash 3
2. Preparação inicial do Git 4
3. Criar um repositório local 4
4. Entender o estado dos arquivos 5
5. Preparar e criar um commit 5
6. Comparar alterações 6
7. Branches 6
8. Merge 6
9. Clonar um repositório 7
10. Conectar um projeto local ao GitHub 7
11. Push 7
12. Pull 8
13. Pull request 8
14. Conventional Commits 9
15. Issues 9
16. Termos comuns do GitHub 11
17. Sequências para consulta rápida 11
18. Erros básicos 12
Referências 13

Este material reúne os comandos básicos que serão utilizados durante o minicurso. A ideia não é decorar todas as linhas. O mais importante é entender o que cada comando faz e consultar `git status` sempre que surgir uma dúvida.

Os exemplos foram escritos para o terminal Bash do Fedora. Os comandos de Git também funcionam no Bash do Linux Mint.

1. Comandos básicos do Bash

O Bash é o terminal no qual vamos executar os comandos. Antes de começar a usar Git, é importante saber entrar em uma pasta, conferir onde estamos e visualizar os arquivos.

Ver a pasta atual

```
pwd
```

`pwd` mostra o caminho completo da pasta em que o terminal está.

Exemplo de resultado:

```
/home/aluno/Documentos
```

Listar arquivos e pastas

```
ls
```

Para mostrar também arquivos ocultos, como a pasta `.git`:

```
ls -la
```

Em `-la`, a letra `l` pede uma lista com detalhes e a letra `a` inclui os arquivos ocultos.

Entrar em uma pasta

```
cd nome-da-pasta
```

Para voltar uma pasta:

```
cd ..
```

Para voltar diretamente à pasta pessoal:

```
cd ~
```

Criar uma pasta

```
mkdir meu-projeto
cd meu-projeto
```

O primeiro comando cria a pasta. O segundo entra nela.

Criar um arquivo vazio

```
touch README.md
```

Também podemos abrir a pasta atual no VS Code:

```
code .
```

O ponto representa a pasta atual.

Mostrar o conteúdo de um arquivo

```
cat README.md
```

Limpar a tela do terminal

```
clear
```

Esse comando limpa apenas a tela. Ele não apaga arquivos nem o histórico do Git.

2. Preparação inicial do Git

Conferir se o Git está instalado

```
git --version
```

O resultado mostrará a versão instalada, por exemplo:

```
git version 2.x.x
```

O número exato pode ser diferente em cada computador.

Configurar nome e e-mail

O Git registra nome e e-mail em cada commit:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

`--global` aplica a configuração ao usuário daquele computador. Em uma máquina compartilhada, pergunte ao responsável antes de alterar essa configuração.

Nome e e-mail identificam a autoria do commit. Eles não são a senha da conta do GitHub.

Para conferir os valores:

```
git config --global user.name
git config --global user.email
```

3. Criar um repositório local

Entre na pasta do projeto e confirme o caminho:

```
cd meu-projeto
pwd
ls
```

Depois, inicie o repositório:

```
git init -b main
```

`git init` cria o repositório. A opção `-b main` define `main` como nome da branch principal.

Se a versão instalada do Git não aceitar essa opção, use:

```
git init
git branch -M main
```

4. Entender o estado dos arquivos

Use este comando com frequência:

```
git status
```

Ele informa:

- qual é a branch atual;
- quais arquivos foram modificados;
- quais arquivos ainda não são acompanhados;
- quais alterações estão preparadas para o próximo commit.

`git status` apenas mostra informações. Ele não altera os arquivos.

5. Preparar e criar um commit

Um commit registra uma versão local do projeto. Salvar o arquivo no VS Code e criar um commit são ações diferentes.

Preparar um arquivo

```
git add README.md
```

Para preparar todos os arquivos modificados da pasta atual:

```
git add .
```

O ponto significa "a pasta atual". Antes de usar `git add .`, confira `git status` para não adicionar algo sem querer.

Criar o commit

```
git commit -m "Adiciona README inicial"
```

A opção `-m` permite escrever a mensagem do commit no próprio comando.

Depois, confira o histórico:

```
git log --oneline
```

`--oneline` mostra cada commit em uma linha.

O fluxo básico fica assim:

```
editar → salvar → git status → git add → git commit
```

6. Comparar alterações

Para ver o que foi modificado antes de executar `git add`:

```
git diff
```

Depois de `git add`, use:

```
git diff --cached
```

`--cached` mostra o que está preparado para o próximo commit.

7. Branches

Uma branch é uma linha de desenvolvimento dentro do mesmo repositório. Ela permite trabalhar em uma mudança sem alterar imediatamente a `main`.

Listar as branches

```
git branch
```

Um asterisco indica a branch atual.

Criar uma branch e entrar nela

```
git switch -c nova-funcionalidade
```

`switch` troca de branch. A opção `-c` cria a branch antes da troca.

Depois de alterar os arquivos, faça o commit normalmente:

```
git status
git add .
git commit -m "feat: adiciona nova funcionalidade"
```

Voltar para a main

```
git switch main
```

Para apenas trocar de branch, não utilize `-c`, pois a branch já existe.

8. Merge

Merge integra as alterações de outra branch à branch atual.

Para integrar `nova-funcionalidade` na `main`:

```
git switch main
git merge nova-funcionalidade
```

A ordem é importante. Primeiro entramos na branch que receberá as alterações. Depois informamos qual branch será integrada.

Confira o resultado:

```
git status
git log --oneline --graph --all
```

`--graph` desenha as linhas do histórico e `--all` inclui todas as branches.

Um conflito pode acontecer quando duas branches alteram a mesma parte de um arquivo de maneiras diferentes. Isso não significa que o repositório foi perdido. Significa que o Git precisa de uma decisão humana antes de concluir o merge.

9. Clonar um repositório

Clone cria uma cópia local de um repositório remoto.

```
git clone https://github.com/USUARIO/REPOSITORIO.git
```

Depois, entre na pasta criada:

```
cd REPOSITORIO
git status
```

Ao clonar, o Git baixa os arquivos e o histórico e configura o remoto com o nome `origin`.

Para conferir:

```
git remote -v
```

10. Conectar um projeto local ao GitHub

Se o projeto começou no computador, crie primeiro um repositório vazio no GitHub. Não adicione README, licença ou `.gitignore` pelo site se esses arquivos já existem localmente.

Copie a URL HTTPS e execute:

```
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git remote -v
```

`remote add` registra um endereço remoto. `origin` é o nome convencional dado ao remoto principal.

11. Push

Push envia commits locais para o repositório remoto.

No primeiro envio da `main`:

```
git push -u origin main
```

A opção `-u` registra a relação entre a branch local e a branch remota. Nos próximos envios, normalmente basta usar:

```
git push
```

O push envia commits. Se uma alteração ainda não foi commitada, ela não será enviada.

Quando a autenticação for solicitada, siga a entrada pelo navegador aberta pelo VS Code. A URL conecta o projeto ao repositório; a autenticação confirma que você possui permissão para enviar alterações.

12. Pull

Pull recebe alterações do repositório remoto e tenta integrá-las à branch atual.

```
git pull
```

Antes de executar, confira:

```
git status
git branch --show-current
```

Use `git pull` quando houver alterações no GitHub que ainda não estão na sua cópia local. Em um trabalho em equipe, isso costuma acontecer quando outra pessoa enviou commits.

Não confunda:

- `git push` envia commits;
- `git pull` recebe e integra commits.

13. Pull request

Pull request não é um comando do Git. É um recurso do GitHub utilizado para propor que as alterações de uma branch sejam integradas a outra.

Um fluxo básico é:

1. Criar uma branch.
2. Fazer as alterações.
3. Criar os commits.
4. Enviar a branch com `git push`.
5. Abrir o repositório no GitHub.
6. Selecionar Compare & pull request.
7. Escrever um título e explicar o que foi alterado.
8. Criar a pull request.
9. Revisar os arquivos modificados.
10. Fazer o merge quando a alteração estiver pronta.

Exemplo no terminal:

```
git switch -c melhora-readme
git add README.md
git commit -m "docs: melhora explicação do projeto"
git push -u origin melhora-readme
```

Depois do push, a criação da pull request é feita no GitHub.

Uma pull request não significa que o código já foi integrado. Ela representa uma proposta que pode receber comentários, revisões e alterações antes do merge.

14. Conventional Commits

Conventional Commits é uma convenção para escrever mensagens de commit de forma mais organizada. Ela não é obrigatória para o Git funcionar. Uma equipe escolhe se deseja utilizá-la.

O formato básico é:

```
tipo: descrição curta
```

Exemplos:

```
feat: adiciona cadastro de alunos
fix: corrige cálculo da média
docs: atualiza instruções do README
refactor: reorganiza função de validação
test: adiciona teste da calculadora
chore: organiza arquivos do projeto
```

Tipos mais comuns:

Tipo	Quando utilizar
`feat`	Nova funcionalidade.
`fix`	Correção de um problema.
`docs`	Alteração apenas na documentação.
`refactor`	Mudança na organização do código sem adicionar funcionalidade ou corrigir um erro específico.
`test`	Criação ou alteração de testes.
`style`	Formatação que não altera o funcionamento, como espaços ou organização visual do código.
`chore`	Tarefa de manutenção ou organização.

Um escopo pode ser colocado entre parênteses, mas ele é opcional:

```
feat(login): adiciona validação de senha
fix(calculadora): impede divisão por zero
```

Para quem está começando, `feat`, `fix` e `docs` já são suficientes. Não complique a mensagem apenas para seguir um padrão. Ela precisa explicar a mudança feita.

15. Issues

Issue é um registro criado no GitHub para organizar algo que precisa ser discutido ou realizado. Pode representar:

- um erro encontrado;
- uma nova funcionalidade;
- uma melhoria;
- uma tarefa;
- uma dúvida sobre o projeto.

Uma issue normalmente possui título, descrição e número. Também pode receber labels, responsável e marco.

Exemplo:

```
Título: Adicionar opção de excluir tarefa

Descrição:
Atualmente é possível criar e listar tarefas, mas não existe uma opção para excluí-las.

Critério de conclusão:
- permitir escolher uma tarefa;
- pedir confirmação;
- remover a tarefa da lista.
```

O GitHub cria uma identificação como `#12`. Esse número pode ser citado em commits e pull requests.

Exemplo de commit relacionado:

```
feat: adiciona exclusão de tarefas
```

Em uma pull request, expressões como `Closes #12` podem fechar a issue automaticamente quando a pull request for integrada. Use isso apenas quando a alteração realmente resolver a issue.

16. Termos comuns do GitHub

Termo	Significado básico
Repositório	Projeto acompanhado pelo Git.
Repositório local	Cópia que está no computador.
Repositório remoto	Cópia acessada por uma URL, normalmente no GitHub.
Commit	Registro de uma versão do projeto.
Branch	Linha de desenvolvimento.
`main`	Nome comum da branch principal.
Merge	Integração de uma branch em outra.
`origin`	Nome convencional do remoto principal.
Clone	Cópia local criada a partir de um repositório remoto.
Push	Envio de commits ao remoto.
Pull	Recebimento e integração de commits do remoto.
Pull request	Proposta de integração feita no GitHub.
Issue	Registro de erro, ideia, tarefa ou melhoria.
Fork	Cópia de um repositório criada em outra conta do GitHub.
Label	Rótulo usado para organizar issues e pull requests.
Assignee	Pessoa responsável por uma issue ou pull request.
Reviewer	Pessoa convidada para revisar uma pull request.
Tag	Nome associado a um ponto específico do histórico.
Release	Versão publicada de um projeto no GitHub.
GitHub Actions	Recurso utilizado para automatizar tarefas, testes e publicações.

17. Sequências para consulta rápida

Criar o primeiro commit

```
git status
git add README.md
git commit -m "docs: adiciona README inicial"
git log --oneline
```

Criar uma branch e integrar na main

```
git switch -c nova-funcionalidade
# alterar os arquivos
git add .
git commit -m "feat: adiciona nova funcionalidade"
git switch main
git merge nova-funcionalidade
```

Enviar um repositório para o GitHub

```
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git push -u origin main
```

Fazer uma nova atualização

```
git status
git add .
git commit -m "docs: atualiza README"
git push
```

18. Erros básicos

`not a git repository`

Você está fora da pasta do repositório. Confira:

```
pwd
ls
```

Depois, entre na pasta correta com `cd`.

`nothing to commit`

Não existe alteração preparada para o commit. Confira se o arquivo foi salvo e execute:

```
git status
git diff
```

`remote origin already exists`

O remoto `origin` já foi configurado. Confira o endereço:

```
git remote -v
```

Não execute `git remote add origin` novamente sem verificar o endereço existente.

Push recusado

Leia a mensagem completa. As causas mais comuns são falta de autenticação, falta de permissão, URL incorreta ou a existência de commits diferentes no repositório remoto.

Não utilize `sudo git`, não apague a pasta `.git` e não execute comandos destrutivos para tentar resolver rapidamente. Pare, consulte `git status` e identifique o estado atual.

Referências

- Documentação oficial do Git
- Sobre o Git - documentação do GitHub
- Pull requests - documentação do GitHub
- Issues - documentação do GitHub
- Conventional Commits 1.0.0 em português

Os comandos foram escritos para o Bash do Fedora e do Linux Mint. Este arquivo foi produzido no Windows e não representa um teste real nos computadores que serão utilizados durante o minicurso.